

Dynamic Gain Adjustment for Multi-Agent AUV Collaboration: A Decision Tree and Genetic Algorithm Approach

Motoki Nakajima

April 30, 2026

Abstract

Autonomous Underwater Vehicles (AUVs) require robust, decentralized control systems to navigate harsh and unpredictable environments. This study models multi-agent AUV collaboration as a 2D multimodal scalar field optimization problem. To overcome the limitations of empirically tuned deterministic rules, which often trap agents in local minima, we propose a Decision Tree-based approach optimized via Genetic Algorithms (GA) to dynamically adjust control gains in real-time. Because existing static datasets cannot capture the dynamic nature of swarm intelligence, we generated a custom synthetic dataset comprising 4,500 trials to train and evaluate our models. We compared the performance of the GA-optimized Decision Tree against a baseline deterministic model and a Random Forest classifier. The results indicate that while the Decision Tree approach significantly accelerates convergence speed, it occasionally struggles to maintain its position at the global minimum—a phenomenon attributed to the GA fitness function prioritizing exploration over stabilization. This study highlights the critical trade-off between convergence efficiency and stability, demonstrating the high interpretability and potential of tree-based models for online multi-agent control, and laying the groundwork for future reinforcement learning applications.

1 Introduction

Autonomous Underwater Vehicles (AUVs) have become increasingly important for various applications, as the demand for underwater exploration and monitoring continues to grow. However, those AUVs often face harsh underwater environments, which we believe to solve by letting multiple AUV agents collaborate with each other. Compared to a centralized control system, a decentralized multi-agent system can be more robust and scalable, especially under dynamic yet uncertain underwater conditions, where absolute measurements are often unavailable. For a measurable experimental evaluation, we simplified the problem into a 2D multi-agent optimization problem, where multiple AUV agents need to collaboratively find the global minimum of a multimodal gaussian scalar field. The initial experiment consisted of multiple agents deterministically moving under four different rules, and the gains of each force were tuned empirically. The results showed that the agents could successfully find the global minimum, but the performance was highly sensitive to the choice of gains, and the agents often got stuck in local minima. To address these issues, we propose to use Decision Trees to dynamically and online adjust the gains of each force based on the current state of the agents and the environment. Combined with Genetic Algorithms, we can optimize the structure and parameters of the decision trees to further improve the performance of the multi-agent system. After the optimization, we compare the performance of the decision tree-based approach with the original deterministic approach, while also comparing with random forest-based approaches. Because static datasets that could be found online are not suitable for our

problem, we will generate synthetic datasets based on the trajectories of the agents under different conditions and rules. The dataset consists of 4,500 trials, where we extracted 5 features from the environmental states and agent states.

2 Exploratory Data Analysis

For the exploratory data analysis, we evaluated the performance of the original deterministic approach under different gain settings and environmental conditions. In total, we ran 30,000 trials among 7 different numbers of agents and random seeds, and we recorded the trajectories of the agents, the final positions, and the corresponding scalar field values at those positions. We attained the results of 97.50% Success rate in escaping from the local minimum, 1656.46 Steps taken in average, and 503.70 Standard Deviation on steps taken.

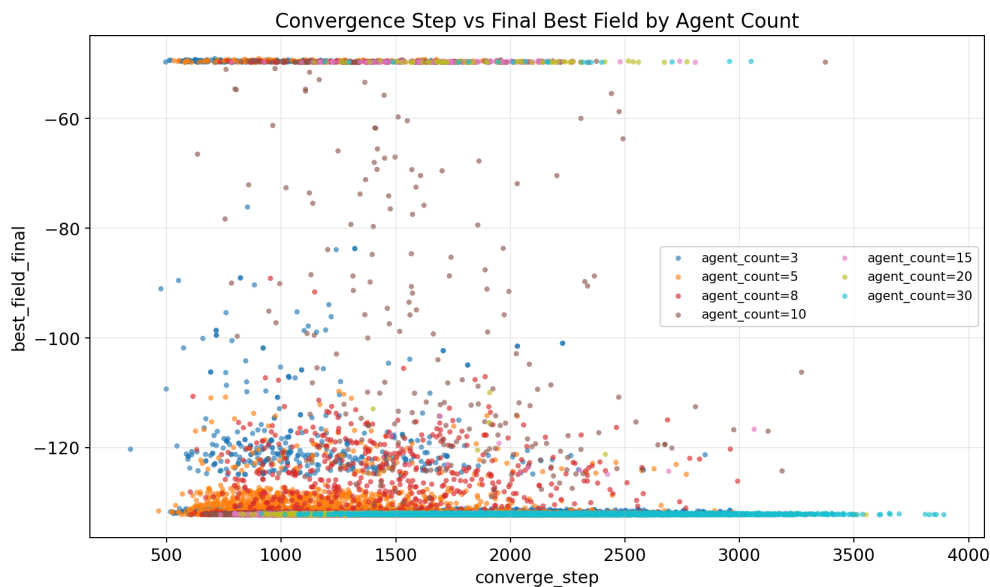


Figure 1: The step taken to converge vs the final best scalar field value.

The figure 1 shows the relationship between the steps taken to converge and the final best scalar field value. We can see that the more agents there are, the more likely the trial will converge to the global minimum, which is consistent with our intuition that more agents can explore the environment more effectively. However, we also observed that there are some trials where the agents took a long time to converge, which indicates that the agents might be getting stuck in local minima or taking inefficient paths to the global minimum.

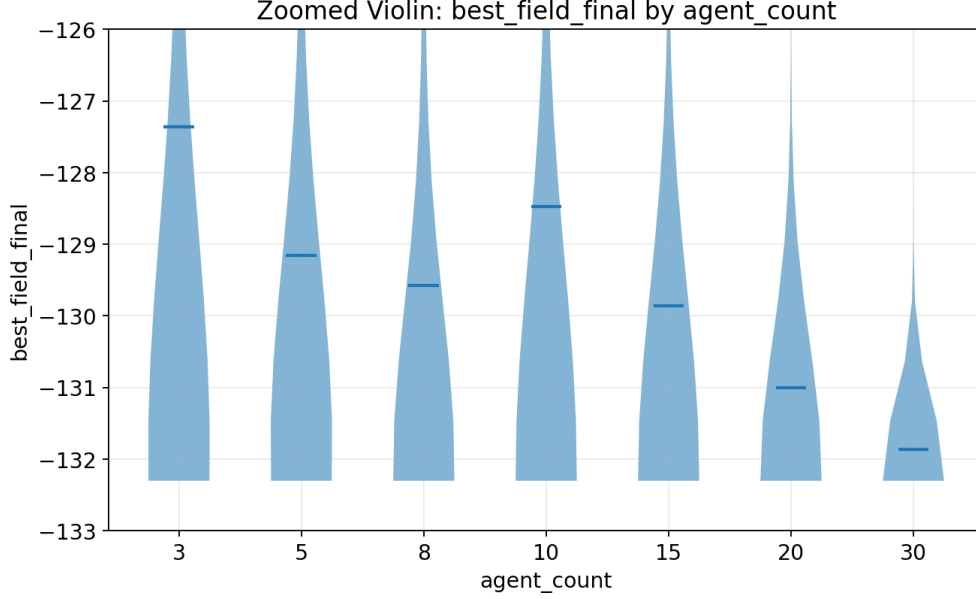


Figure 2: The distribution of the final best scalar field value for different numbers of agents.

The figure 2 shows the distribution of the final best scalar field value for different numbers of agents. This visually confirms that as the number of agents increases, the distribution of the final best scalar field value becomes more concentrated around the global minimum, which further supports our observation from the previous figure.

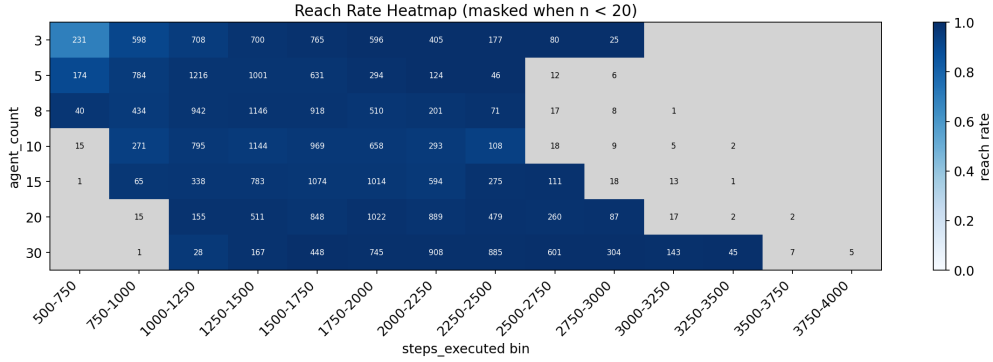


Figure 3: The heatmap of the final best scalar field value and steps taken.

The figure 3 shows the heatmap of the final best scalar field value and steps taken. The darker the cell is, the more likely the trial will converge to the global minimum. Considering the calculation cost of the decisions, which increases as it goes to the right-bottom, we can see that the optimal choice of the number of agents is around 5, which balances the performance and the computational cost. This insight is supported under the visual analysis of the previous two figures, where we can point out a dent in the right side of the heatmap, which indicates that the performance does not improve significantly after a certain number of agents.

This exploratory data analysis included the empirical evaluation of the original deterministic gains.

1. $k_{avoid} = 2.0$

2. $k_{quark} = 0.6$

3. $k_{dir} = 3.5$

4. $k_{drag} = 0.25$

Regarding data quality, since the dataset was synthetically generated via a deterministic physics simulator, the resulting dataset was clean and complete. Therefore, there were no missing or null values, and no data imputation techniques were required. Furthermore, the extracted input features (such as agent speed, neighbor count, and nearest neighbor distance) were normalized to ensure consistent scales before moving to the model training phase.

3 Methodology

3.1 Feature Engineering

Each agents were granted access to the following features:

1. Agent Kinematics: Speed, Acceleration, and Heading.
2. Environmental Perception: Local Scalar Field Value.
3. Agent Memory: Previous Local Scalar Field Value.
4. Communicated Information: Neighboring Agents' features that are available within themselves.

Those features were chosen due to their limited access to information, which is more realistic in real-world underwater environments, where absolute measurements are often unavailable. Combined with these features, the decision tree-based approach can dynamically adjust the gains of each force based on the current state of the agents and the environment. The 4 forces are deterministically calculated based on the current state of the agents and the environment, and they are as follows:

1. Avoidance Force: Forces that repel agents from each other, which prevents collisions and promotes exploration.

$$\mathbf{d}_{ij} = \mathbf{x}_j - \mathbf{x}_i, \quad r_{ij} = \|\mathbf{d}_{ij}\|$$

$$\mathbf{a}_{\text{avoid},i} = k_{\text{avoid}} \sum_{j \neq i, r_{ij} < R_{\text{avoid}}} \left(-\frac{\mathbf{d}_{ij}}{r_{ij} + 0.1} \right)$$

2. Quark Force: Forces that attract agents towards each other, which prevents them from dispersing too much and encourages collaboration.

$$\hat{\mathbf{d}}_{ij} = \frac{\mathbf{d}_{ij}}{r_{ij}}, \quad s_{ij} = \frac{r_{ij}}{r_{ij} + R_{\text{sat}}}$$

$$\mathbf{a}_{\text{quark},i} = k_{\text{quark}} \sum_{j \neq i} \left(F_{\text{max}} s_{ij} \hat{\mathbf{d}}_{ij} \right)$$

3. Directional Force: Forces that guide agents towards the direction of the steepest descent of the scalar field, which helps them to find the global minimum more efficiently.

$$\Delta \mathbf{x}_i(t) = \mathbf{x}_i(t) - \mathbf{x}_i(t-1), \quad \Delta f_i(t) = f_i(t) - f_i(t-1)$$

$$\hat{\nabla} f_i^{\parallel}(t) = \frac{\Delta f_i(t)}{\|\Delta \mathbf{x}_i(t)\|^2 + \varepsilon} \Delta \mathbf{x}_i(t)$$

$$\mathbf{a}_{\text{dir},i}(t) = -k_{\text{dir}} \hat{\nabla} f_i^{\parallel}(t)$$

4. Drag Force: Forces that simulate water resistance, which slows down the agents and helps them to settle into the global minimum.

$$\mathbf{a}_{\text{drag},i}(t) = -k_{\text{drag}} \mathbf{v}_i(t)$$

3.2 Model Selection & Hyperparameter Tuning

In this experiment, we chose to use two types of decision making models: Decision Trees and Random Forests, and an empirically tuned deterministic approach as the baseline.

The empirically tuned deterministic approach uses fixed gains for the four forces, which were determined through trial and error based on the performance of the agents in the environment. The performance of this approach was evaluated in the exploratory data analysis section, and will be used as a benchmark to compare against the decision tree-based approaches.

The Decision Tree-based approach uses a single decision tree to dynamically adjust the gains of each force based on the current state of the agents and the environment. The decision tree are allowed to input the following environmental inputs:

1. Agent Speed: The current speed of the agent.
2. Agent Acceleration: The current acceleration of the agent.
3. Scalar Gradient Magnitude: The magnitude of the estimated gradient of the scalar field at the agent's current position.
4. Neighbor Count: The number of neighboring agents within a certain radius.
5. Nearest Neighbor Distance: The distance to the nearest neighboring agent.
6. Neighbor Average Distance: The average distance to neighboring agents within a certain radius.

These inputs were then fed to the decision tree, which the output would be the increments of the gains for each of the four forces, which would then be added to the original empirically tuned gains to get the final gains for each step. The decision tree was picked due to its interpretability and low computational cost, which makes it suitable for online decision making in a multi-agent system. The tree's depth was strictly limited to 3 to prevent overfitting and to ensure the model's transparency, which is crucial for understanding the decision-making process of the agents and for debugging purposes.

The Random Forest-based approach uses an ensemble of decision trees to make the same adjustments to the gains of each force, which can potentially improve the performance by reducing

overfitting and increasing the robustness of the model. However, considering the computational cost of the random forest-based approach in an online setting, we will only compare the performance of the random forest-based approach in an offline setting, where we can afford the computational cost of making decisions based on an ensemble of trees. We used successful agent decision making data from other trials as target labeled data to train the random forest model, exploring the potential of using an offline trained model to make online decisions.

The decision tree-based approach included a genetic algorithm optimization step, where we optimized the structure and parameters of the decision tree to further improve the performance of the multi-agent system. The genetic algorithm was picked due to its ability to optimize complex, non-differentiable functions, which is suitable for optimizing the structure and parameters of a decision tree. The hyperparameters used in the genetic algorithm optimization included:

1. Population Size (= 30): The number of candidate solutions in each generation.
2. Mutation Rate (= 0.35): The probability of randomly mutating a candidate solution.
3. Crossover Rate (= 0.9): The probability of combining two candidate solutions to create a new candidate solution.
4. Number of Generations (= 30): The number of iterations for the genetic algorithm to run.

Each candidate evaluated its fitness by running 100 trials (10 trials x 10 seeds) of the multi-agent system using the gains determined by the decision tree, and the fitness was calculated based on the average final best scalar field value and the average steps taken to converge, with a higher weight on the final best scalar field value to prioritize finding the global minimum.

3.3 Train/Test Split

The training and testing data were completely separated by utilizing the field seeds, ensuring that the model was evaluated on unseen environments. The training set consisted of 90,000 trials, which were used to train the decision tree and random forest models, as well as to perform the genetic algorithm optimization for the decision tree. The testing set consisted of 4,500 trials under a fixed field with random initial conditions, which were used to evaluate the performance of the trained decision tree and random forest models, as well as to compare against the baseline empirically tuned deterministic approach.

4 Results

The results consist of the performance of the decision tree-based approach and the random forest-based approach compared to the baseline empirically tuned deterministic approach.

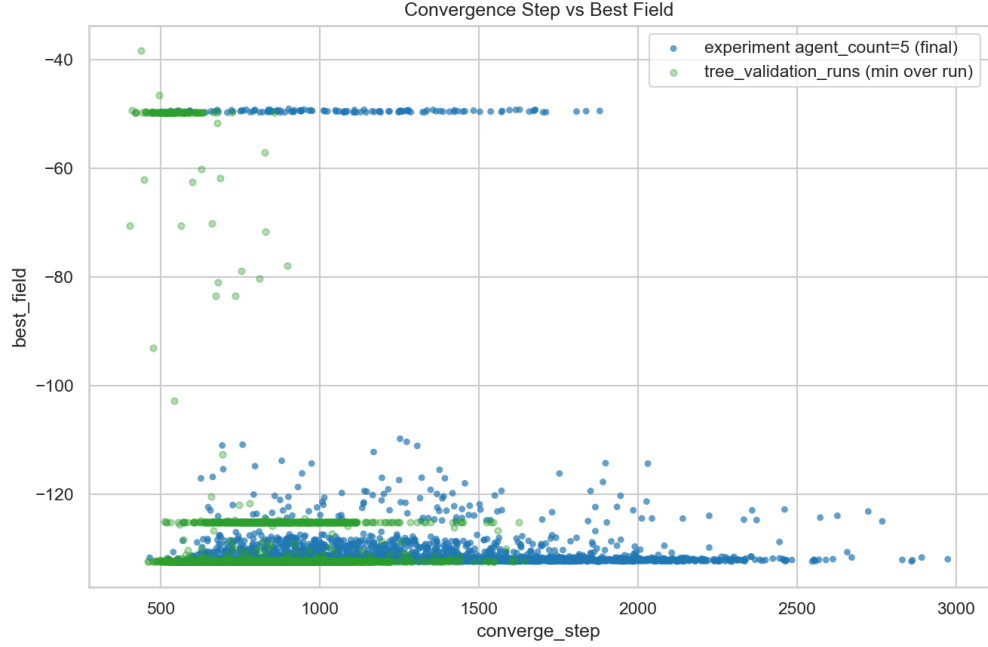


Figure 4: The scatter plot of the best scalar field value over the run between the decision tree-based approach and the baseline empirically tuned deterministic approach.

The figure 4 shows that the tree-based approach converges earlier as the cluster of the green dots are relatively more on the left side of the plot.



Figure 5: The scatter plot of the best scalar field value between the baseline, the tree-based when converged, and the tree-based minimum over the run.

The figure 5 shows the comparison of the best scalar field value between the baseline, the tree-

based approach when it has converged, and the tree-based approach at its minimum over the run. The orange dots representing the final values place way higher than both the blue and the green dots, which indicates worse performance when the tree-based approach settles to a convergence.

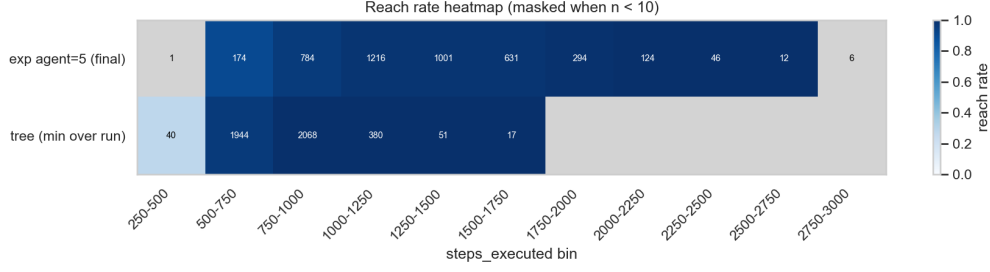


Figure 6: The heatmap of the final best scalar field value and steps taken.

The figure 6 shows the heatmap of the final best scalar field value and steps taken for the tree-based approach. The heatmap shows that the tree-based approach has a much less steps taken to converge compared to the baseline, which could be seen by the coloured region being more on the left side of the plot.

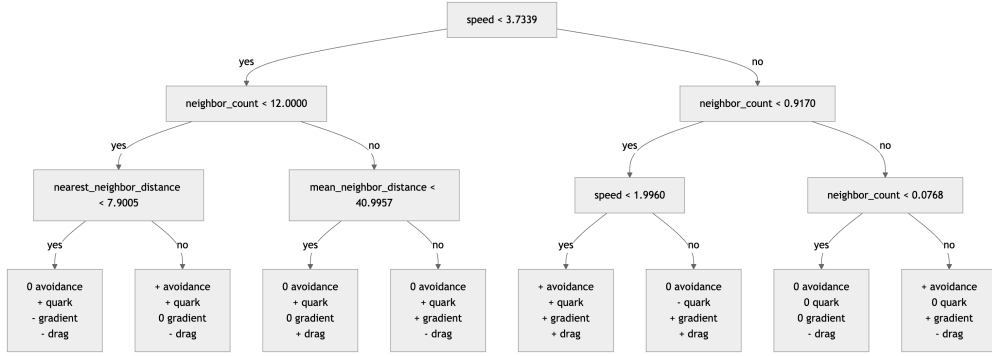


Figure 7: The trained decision tree after the genetic algorithm optimization.

The figure 7 shows the trained decision tree after the genetic algorithm optimization. The decision tree shows that the most important feature for adjusting the gains is the speed, as the first split is based on the speed of the agent.

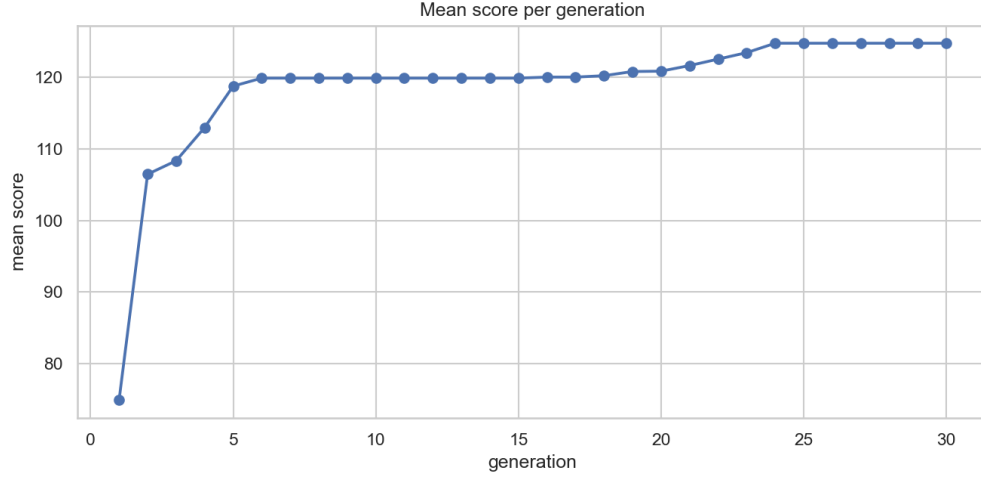


Figure 8: The improvement of the fitness score over the generations in the genetic algorithm optimization.

The figure 8 shows the improvement of the fitness score over the generations in the genetic algorithm optimization. The fitness score shows a general increasing trend over the generations with roughly 2 major breakthroughs, while other places remained relatively flat.

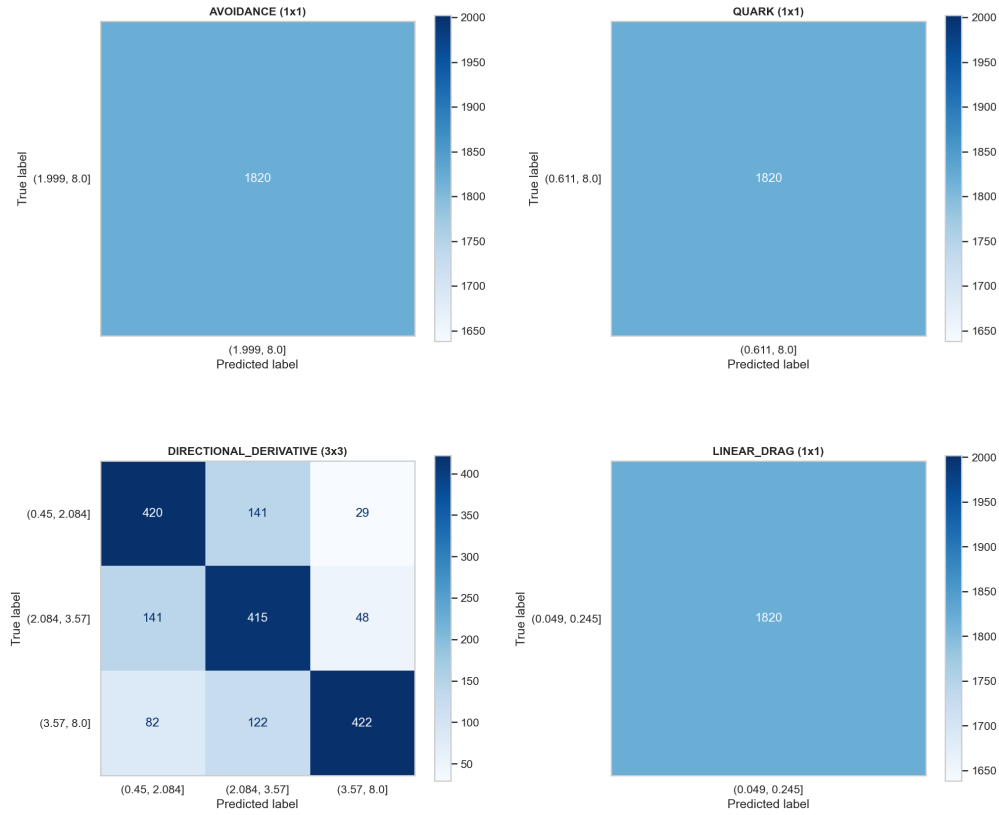


Figure 9: The confusion matrix of the random forest-based approach on the testing set.

The figure 9 shows the confusion matrix of the random forest-based approach on the testing

set. The confusion matrix shows that the random forest-based approach has a high true positive rate.

Gain	Classes	Accuracy	Balanced Acc.	F1	Precision	Recall	RMSE
avoid	1	1.0000	1.000	1.000	1.0000	1.000	0.0000
quark	1	1.0000	1.000	1.000	1.0000	1.000	0.0000
dir	3	0.6907	0.691	0.693	0.7037	0.691	0.7016
drag	1	1.0000	1.000	1.000	1.0000	1.000	0.0000

Table 1: Performance metrics of Random Forest models for each gain prediction task.

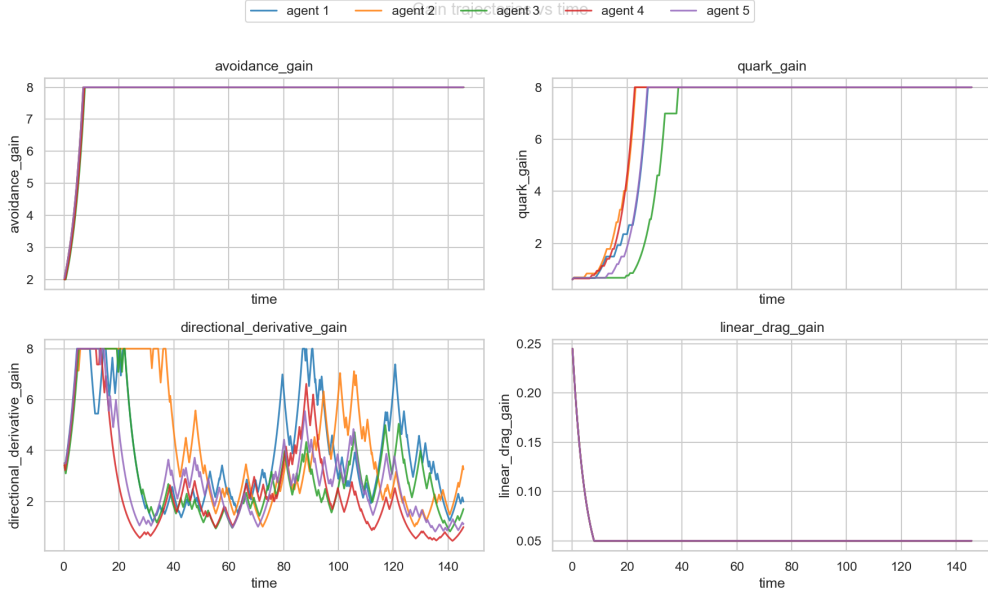


Figure 10: The gain adjustments over time for a sample trial using the decision tree-based approach.

The figure 10 shows the gain adjustments over time for a sample trial using the decision tree-based approach. While the three gains, avoidance, quark, and drag, remain extremely stabilized at their limits, the directional gain shows a lot of adjustments over time.

5 Discussion & Conclusion

5.1 Performance vs Calculation Cost Trade-off

The decision tree-based approach showed a significant improvement in the performance of the multi-agent system compared to the baseline empirically tuned deterministic approach, as evidenced by the earlier convergence and better final scalar field values. However, the decision tree-based approach converges to a worse final scalar field value compared to the baseline, which indicates that while the decision tree can help the agents to converge faster, it might also lead them to settle in local minima more often than the baseline. This trade-off between performance and convergence speed is an important consideration when designing multi-agent systems, especially in real-world applications where computational resources and time are limited. Considering the computational cost of the deterministic approach, which costs a lot more steps to converge, the decision tree-based

approach can be a more efficient choice in scenarios where quick convergence is more critical than achieving the absolute best performance.

5.2 Model Interpretability

The decision tree-based approach offers a high level of interpretability, as the structure of the decision tree can be easily visualized and understood. The trained decision tree after the genetic algorithm optimization showed that the most important feature for adjusting the gains is the speed of the agent, which provides insights into how the agents are making decisions based on their current state and the environment. This interpretability can be valuable for debugging and improving the multi-agent system, as it allows us to understand the decision-making process of the agents and to identify potential areas for improvement. Additionally, we can observe from 10 that the directional gain is the only gain that shows a lot of adjustments over time, which indicates that the decision tree is primarily adjusting the directional gain to guide the agents towards the global minimum, while keeping the other gains completely stable. On the other hand, the random forest-based approach, while potentially offering better performance due to its ensemble nature, lacks the same level of interpretability as the decision tree-based approach, which can make it more difficult to understand and debug. With the simplicity of the inputs and outputs, the decision tree-based approach can be a more suitable choice for applications where interpretability is a key requirement.

5.3 GA Optimization

The genetic algorithm optimization for the decision tree showed a general increasing trend in the fitness score over the generations, which indicates that the optimization process was effective in improving the performance of the decision tree-based approach. The two major breakthroughs in the fitness score suggest that there were significant improvements in the decision tree structure or parameters at those points, which could be further analyzed to understand what changes led to those improvements. Because the algorithm rewarded the fitness based on the best field value over the run, the model was optimized to find but not converge to the global minimum, which could be a reason for the worse performance of the decision tree-based approach when it converges. The outputs imply that the trained decision tree was capable of bravely exploring the environment to find the global minimum, yet it lacked the ability to know that they have explored enough.

5.4 Future Work

Future work could explore the following directions:

1. Incorporating additional features into the decision tree, such as more detailed environmental perceptions or more complex agent memory, which could potentially improve the performance of the decision tree-based approach.
2. Exploring other types of models for dynamic gain adjustment, such as neural networks or reinforcement learning-based approaches, which could potentially offer better performance at the cost of interpretability.
3. Investigating the trade-off between performance and convergence speed in more detail, and developing strategies to balance this trade-off based on the specific requirements of the application.

4. Analyzing the decision tree structure and parameters after the genetic algorithm optimization to gain insights into what changes led to the improvements in performance, and using these insights to guide future optimizations.
5. Simulating more complex and realistic underwater environments to evaluate the performance of the decision tree-based approach in scenarios that are closer to real-world applications.

References

- [1] ISO/IEC. (2020). *ISO/IEC 14882:2020 Programming languages — C++*. Available at: <https://en.cppreference.com/>
- [2] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- [3] Fortin, F.-A., et al. (2012). DEAP: Evolutionary Algorithms Made Easy. *Journal of Machine Learning Research*, 13, 2171-2175.
- [4] T. Maki, et al. (2012). A method for wide area survey of seafloor using multiple autonomous underwater vehicles. *2012 IEEE/OES Seventh International Symposium on Underwater Technology (UT)*, 1-5.
- [5] Motoki Nakajima. (2026). *Multi-Agent AUV Optimization Simulator [Source Code]*. GitHub repository. Available at: https://github.com/motokinakajima/uuv_sim